TECHNICAL VERIFICATION REPORT

Moh Maya V1 Foundation

Verification of the current implementation against Bible V1.0, World & Engine Definition V1.0, Resources & States V1.0, and Entity Data Model V1.0, plus the agreed pause-between-decisions architecture.

Source: live codebase at D:\coding_world\Moh_Maya. Inspected 17 September 2026. Application code was not modified for this review.

Metric	Value
pytest cases passing	15
HTTP endpoints	8
PostgreSQL tables	15
LLM / background simulation loop	0

Verdict. The V1 foundation exists and is coherent: FastAPI inspects, SimulationEngine advances one world inside a SQLAlchemy session, PostgreSQL is the write path, the graph only reads, and the world returns to paused. Remaining gaps are stub depth, hardcoded Laketown behavior, and an unenforced knowledge boundary on God APIs.

1. Current implementation

1.1 What has been implemented

The repository is a Python modular monolith. FastAPI is the HTTP layer. The simulation core is HTTP-independent. Schema is created only by Alembic (the app never calls create_all). Docker Compose provides PostgreSQL 16. A seed script creates one development world, Laketown. Tests run against that Postgres instance.

1.2 Project structure

- **app/main.py** — FastAPI factory; documents the API as a developer/God surface, not a player surface.
- **app/core** — pydantic-settings from .env; SQLAlchemy SessionLocal; autocommit=False.
- **app/api/routes** — health, world, people, locations, simulation.
- **app/models** — SQLAlchemy 2.0 ORM with UUID keys and JSONB for evolving resources/states.
- **app/repositories** — world_id required on get/list for world-owned entities.
- **app/services** — WorldService (inspect), SimulationService (decision application).
- **app/simulation** — SimulationEngine plus World, Character, Event, Health, Economy, Relationship, Knowledge, Government engines, and ActionResolver.
- **app/graph/service.py** — read-only SQL → {nodes, edges}.
- **alembic/versions/001_initial_schema.py** — single initial migration.
- **scripts/seed_world.py** — Laketown seed; --reset recreates it.
- **tests/** — 15 pytest cases.

1.3 Database tables / models

Table	Role
worlds	Root clock: current_day, current_minute, status, tick_minutes, calendar, world_state, history, player_person_id
locations	Places with parent_location_id; state/capacity/properties JSONB

Table	Role
people	current_location_id + household_id; states/resources/schedule JSONB
households	Residence unit + resources/states/obligations
organizations	Institution stub (Municipal Council)
businesses	Cafe, Clinic + inventory JSONB
infrastructure	Municipal Water Works
natural_entities	Laketown Lake quantity/limit
items	Cafe Keys, Arjun's Phone
events	Causal history (origin + specific_type + time + consequences)
event_participants	Who participated and in what role
relationships	Polymorphic typed edges (CONTAINS, LOCATED_AT, …)
knowledge_records	world_truth / person / player layers
memories	Person-associated remembered experiences

1.4 API endpoints

Method	Path
GET	/health
GET	/world/{world_id}
GET	/world/{world_id}/locations
GET	/world/{world_id}/people
GET	/world/{world_id}/events
GET	/world/{world_id}/relationships
GET	/world/{world_id}/graph
POST	/world/{world_id}/simulation/advance

There is no list-worlds, create-world, knowledge, or memory endpoint. All inspect routes return authoritative world state. They are labelled as a developer/God surface, not a player knowledge surface.

1.5 Simulation flow

The world stays paused until a player decision is posted. ActionResolver maps WAIT or WALK_TO_CAFE to a game-minute cost. SimulationEngine converts that cost into N ticks (ceil(cost / tick_minutes)), runs engines once per tick, applies pending player effects, sets status back to paused, and returns a summary. There is no while-True loop, cron, or background job.

1.6 Seeded world / entities

Laketown starts at Day 1, 08:00 (minute 480), status paused, tick_minutes = 1. Locations: Town Square, Cafe, Hospital, Residential Area. People: Arjun (player), Sanjay, Priya, Ravi, Meera, Asha, Dr. Nandini, Vikram. Households: Sharma, Kumar, Arjun. Businesses: Laketown Cafe (closed), Laketown Clinic (open). Also: Municipal Council, Water Works, Lake, Cafe Keys, Arjun's Phone, a MorningBegan God event, and three knowledge layers (world truth that Asha is considering leaving Ravi; Ravi knows Asha is unhappy; Arjun knows there is a cafe).

1.7 Graph implementation

GraphService.build(world_id) reads SQL and returns JSON. It never writes. Nodes are the world plus all entity tables. Edges come only from the relationships table; dangling endpoints are skipped. Knowledge and memory rows are not graph nodes, which matches the Entity Data Model rule that those concepts should not automatically become nodes. Removing the graph layer does not affect simulation.

2. Simulation lifecycle

Trace of **POST /world/{world_id}/simulation/advance** with body {"action_type": "WALK_TO_CAFE"} or {"action_type": "WAIT", "minutes": N}.

Step	Layer	What happens
1	FastAPI route	app/api/routes/simulation.py parses PlayerDecision, opens get_db() session, calls SimulationService.advance. No tick math in the route.
2	Application service	WorldService.get_world(world_id) raises 404 if missing. Then SimulationEngine.advance_decision.
3	ActionResolver	WALK_TO_CAFE → 30 game minutes + effects [move_player_to Cafe, increase_player_fatigue 1]. WAIT → N minutes, no effects.
4	Tick count	ticks = ceil(time_cost_minutes / world.tick_minutes). Default tick_minutes=1 → 30 ticks.
5	Engine.advance	status=advancing; flush. For each tick: add tick_minutes to clock, then run engines. Clock advances before work, so 08:00 itself is never processed.
6	Engines per tick	World conditions/constraints/water → Character schedules → Event pending → Health → Economy → Relationship → Knowledge → Government (no-op).
7	Pending effects	After all ticks: move player to Cafe, bump player fatigue, record PersonMovedLocation.
8	Pause + persist	status=paused; flush. Engine does not commit. get_db() commits on success and rollbacks on any exception.

How 30 minutes works: start current_minute = 480. Each tick does current_minute += tick_minutes (1), wrapping at 1440 into the next day. After 30 ticks: 510 = Day 1, 08:30. Schedule matches use the post-advance minute, so Priya fires at 490 (08:10), Sanjay at 495 (08:15), Meera at 500 (08:20). Ravi's work entry at 540 (09:00) is not reached.

3. World state after a 30-minute WALK_TO_CAFE

The following values change when Laketown is advanced with WALK_TO_CAFE from Day 1, 08:00. Observed both in engine tests and in a live HTTP advance of the seeded world.

What	Before	After	Writer
world.current_minute / clock	480 / Day 1, 08:00	510 / Day 1, 08:30	SimulationEngine
world.status	paused	paused (advancing only mid-transaction)	SimulationEngine
Arjun.current_location_id	Town Square	Cafe	pending effect move_player_to
Arjun.states.fatigue.level	2	3, note=walk	pending effect
Sanjay location + cafe state	Residential; cafe closed	Cafe; operational=open	CharacterEngine.open_cafe at 08:15
Priya.states.fatigue.level	3	2, note=morning_routine	CharacterEngine at 08:10
Meera	waiting for a call	PhoneCallReceived + knowledge + memory	Character + Knowledge engines at 08:20
Ravi	home; work at 09:00	unchanged location	schedule 540 not reached
Laketown Lake.quantity	80	79 (once per advance)	WorldEngine.consume_placeholder_water

What	Before	After	Writer
Laketown Cafe inventory.coffee	12	11 after open	EconomyEngine when operational=open
Laketown Clinic	open, supplies 20	morning_prep_applied=true; supplies unchanged	EconomyEngine on first tick (clinic already open)
Events	MorningBegan only	+ PersonMorningRoutine, PersonMovedLocation (Sanjay), CafeOpened, PhoneCallReceived, PersonMovedLocation (Arjun)	context.record_event

4. Persistence

World state survives API restart — PASS. Authoritative fields are ORM columns and JSONB. There is no in-memory world store. Reloading via a new SQLAlchemy session reads the committed clock. A dedicated uvicorn-restart test does not exist; a new SessionLocal is the equivalent check if PostgreSQL is the store.

Simulation changes are persisted — PASS. After a successful advance, get_db() commits. Subsequent GET /world/{id} and GET /people return the new clock, locations, and states. test_state_persistence_across_sessions asserts current_minute == 487 after a 7-tick WAIT in a fresh session.

Transactions are used correctly — PASS. SessionLocal is created with autocommit=False. SimulationEngine only flush()es. Commit is solely in get_db() after the route returns. Exception → rollback → close.

Failed advances roll back completely — PASS. test_transaction_rollback_on_simulation_failure injects RuntimeError after tick 1. Clock stays 480, status paused, no events. On the HTTP path, RuntimeError is not caught by the route, so get_db()'s except path still rollbacks.

5. World isolation

Every world-owned table has world_id with ON DELETE CASCADE. Person, Location, and Event repositories require world_id on both get and list. test_world_isolation creates World A and World B, each with a person named Sanjay, and asserts World A cannot list or get World B's people, locations, or events.

Gaps in the test, not necessarily in the queries: isolation is untested for businesses, households, relationships, knowledge, memories, graph, and simulation.advance. worlds.name is globally unique, so two worlds cannot share the name Laketown. locations.name is not unique per world in the schema; location_by_name is first-match among one world's loaded rows.

6. Entity model vs Entity Data Model V1.0

V1 entity	Implemented	Notes
World	Yes	Time split into current_day + current_minute. Extra: status, tick_minutes, player_person_id (no FK).
Location	Yes	type stored as location_type. People are not child rows.
Person	Yes	Extra household_id FK and schedule JSONB. Relationships/knowledge/memory are tables, not columns.
Household	Yes	members via LIVES_IN / MEMBER_OF rows
Organization	Yes	No locations column; members via relationships
Business	Yes	owners/employees via OWNS / WORKS_AT
Infrastructure	Yes	dependencies JSONB + DEPENDS_ON row

V1 entity	Implemented	Notes
Natural Entity	Yes	capacity_quantity JSONB
Item	Yes	polymorphic owner_entity_type/id
Event	Yes	origin + specific_type; participants table

Extra tables that are not extra V1 entity types: relationships, event_participants, knowledge_records, memories. The Entity Data Model says knowledge, memory, and relationships should not automatically become graph nodes; GraphService does not emit knowledge/memory nodes. Missing relative to EDM field groups, not missing entity types: Organization.locations as a first-class field; World.current_time as a single datetime; Person.relationships as an embedded field (intentionally relational).

7. Simulation architecture

Rule	Evidence	Status
FastAPI is not responsible for simulation logic	Route only calls SimulationService; tick loop is in SimulationEngine	PASS
Simulation does not depend on FastAPI	No fastapi import under app/simulation, app/models, app/repositories, app/graph	PASS
PostgreSQL is authoritative	SQLAlchemy ORM + Alembic; GraphService has no writes	PASS
Graph is derived / non-authoritative	Read-only nodes/edges from SQL	PASS
LLM not involved	requirements.txt has no LLM stack; no OpenAI/Ollama/LangChain in app/	PASS
No background / offline simulation	No while-True tick loop, no cron, no BackgroundTasks in app/	PASS
World paused between decisions	status set paused before return; no auto-continue	PASS
Hidden truth not auto-exposed to the player	Three-layer tables exist; all HTTP routes are God inspect	PARTIAL

8. Tests

Test	What it verifies
test_database_connection	SELECT 1 against PostgreSQL
test_world_creation	World row, 08:00, paused
test_location_creation	Location insert and world-scoped get
test_person_creation	Person insert; current_location_id set
test_household_creation	Household insert; location_id set
test_business_creation	Business insert; location_id set
test_relationship_creation	LOCATED_AT row found by world_id + from + type
test_event_creation	MorningBegan listed for that world
test_world_isolation	World A cannot list/get World B people, locations, events
test_world_time_advancement	10 ticks: 08:00 → 08:10, paused
test_simulation_advancement_walk_to_cafe	30 ticks; cafe opens; Priya fatigue; Arjun at Cafe
test_state_persistence_across_sessions	New Session sees committed minute 487
test_transaction_rollback_on_simulation_failure	Injected fail after 1 tick leaves clock 480
test_graph_representation_generation	World/location/person nodes + LOCATED_AT edge
test_health_endpoint	GET /health → {status: ok}

Important V1 requirements with no test coverage: HTTP POST advance; HTTP 404 for missing world; knowledge/memory persistence; seed_world.py; graph after a real advance; isolation of relationships/businesses/knowledge; dedicated test database. Tests currently use the application DATABASE_URL. Tests that commit leave extra worlds in the Laketown database.

9. Deviations

Status values are only PASS, PARTIAL, or MISSING.

Specification	Expected	Current implementation	Status
Bible — world does not revolve around the player	NPCs act without the player	NPC schedules fire during the player's elapsed ticks; world does not run while the player is away	PARTIAL
Bible / Engine — player only knows what they know	Player APIs hide world truth	Three-layer tables exist; inspect APIs return full state	PARTIAL
Bible V1 — world continues when player not involved	Offline continuation	Intentionally paused (architecture decision overrides Bible here)	MISSING
Bible V1 — LLM conversations	LLM-driven interaction	Explicitly not integrated	MISSING
Engine — tick + engine boundaries	Orchestrated engines	SimulationEngine calls 8 engines then pauses	PASS
Engine — Character Engine autonomy	People act from circumstances	Deterministic JSON schedules; hardcoded Cafe / Meera names	PARTIAL
Engine — Event Engine scheduled/emergent/God	Events from calendar, conditions, God	CharacterEngine fires demo schedules; EventEngine only resolves PENDING rows (none seeded); God only in seed	PARTIAL
Engine — constraint pressure	Behavior changes before hard limits	Placeholder used/limit and quantity/limit ratios written to JSONB; no downstream behavior	PARTIAL
Engine — World Engine rejects impossible changes	Invalid transitions fail	Only missing Cafe location is rejected on WALK_TO_CAFE	PARTIAL
Engine — Government & Society	Laws, taxes, elections (lean)	government_engine.process_tick is pass	MISSING
Resources & States — categories not entities	JSONB / associated data	Resources and states live on entity JSONB	PASS
Resources & States — no locked formulas	Defer numbers	Demo uses ±1 fatigue, −1 water, −1 coffee, ratio pressure bands	PARTIAL
Resources — Energy vs Fatigue distinct	Energy resource vs fatigue state	Both seeded; walk updates fatigue only	PARTIAL
EDM — 10 entity types	World through Event	All 10 present plus relationship/knowledge tables	PASS
EDM — relationship vocabulary	CONTAINS … SERVES	All 12 types seeded and stored in PostgreSQL	PASS
EDM — people are not location children	current location, movable	current_location_id + LOCATED_AT row updated on move	PASS
Arch — multi-world one database	world_id on owned rows	Implemented; one seed world; unique world.name globally	PASS
Arch — Alembic not create_all	Migrations only	001_initial; app startup does not create tables	PASS

Intentionally out of scope for "missing" LLM/offline/frontend items: they are forbidden by the V1 implementation decisions even where Bible V1 mentions them.

10. Problems / risks found in this code

Only defects visible in the current implementation. Hypothetical future problems are omitted.

- **KnowledgeEngine** looks up the recipient with person.name == "Meera". A phone-call schedule on any other person writes knowledge to Meera or to nobody.

- **CharacterEngine** and **ActionResolver** hardcode location/business names "Cafe" and "Laketown Cafe". A second world without those names cannot run the demo actions correctly.
- **RelationshipEngine**, on PhoneCallReceived, mutates the first KNOWS row in that world (list order), not a relationship involving the call's participants. In Laketown that may stamp Ravi→Asha instead of Meera→Sanjay.
- **EconomyEngine** treats every business with states.operational == "open" as needing morning prep. Laketown Clinic starts open, so on tick 1 it sets morning_prep_applied and logs inventory use even though it has no coffee key and supplies are unchanged.
- **worlds.player_person_id** has no foreign key to people.id.
- **locations.name** is not unique per world in the schema; location_by_name is first match.
- Tests use the application DATABASE_URL. Tests that commit leave extra worlds in the Laketown database.
- POST /simulation/advance is not covered by an HTTP test; only the engine is.
- God APIs have no knowledge/memory inspect routes, so the three information layers cannot be verified through the documented developer surface.

11. Recommended next steps before LLM

These steps stay inside the agreed V1 architecture. Do not add an LLM until the information boundary and schedule/event wiring are entity-id based.

#	Step	Why before LLM
1	Player-perception read API: return only knowledge.layer=player / that person_id's memories and observed events	Bible hard rule. An LLM on God APIs would leak world truth.
2	Replace hardcoded names (Cafe, Laketown Cafe, Meera) with entity ids on schedule entries and action effects	LLM-proposed actions must be validated against ids, not demo strings.
3	Move pending/scheduled occurrences into EventEngine (PENDING rows) and keep CharacterEngine as activity execution	Engine Definition: Event Engine owns what occurs; Character Engine owns what a person does.
4	God inspect for knowledge_records and memories; list worlds; stop tests from committing into the Laketown DB	Cannot debug the information boundary or isolation if those surfaces and tests are missing.
5	Fix RelationshipEngine to update participants of the actual event, and stop EconomyEngine from tagging unrelated open businesses	Causal records must be true before an LLM narrates them.

End of report. This document verifies what exists against the already agreed V1 architecture. It does not propose a redesign.